CSCI-GA 3033: Building LLM Reasoners, Spring 2026

Reasoning RL

Aditeya Baral
N19186654 — ab12057

Problem 1: Zero-shot MATH Baseline

Part (a): Category Analysis

Cat.	Description	Count	%
1	Format reward = 1, Answer reward = 1 (correct)	304	60.8%
2	Format reward = 1, Answer reward = 0 (wrong answer)	166	33.2%
3	Format reward = 0, Answer reward = 0 (no `\boxed{}`)	30	6.0%
	Total	500	100%

Table 1: Breakdown of 500 MATH test examples by reward category for Qwen 2.5 Math 1.5B (zero-shot, $T = 0$, max 2048 tokens).

Solution

Category 1 example (format = 1, answer = 1):

Problem: Convert the point $(0, 3)$ in rectangular coordinates to polar coordinates.

Response (tail): ... Python confirms $r = 3$, $\theta = \pi/2$. $\boxed{\left(3, \frac{\pi}{2}\right)}$

Ground truth: $\left(3, \frac{\pi}{2}\right)$ ✓

Category 2 examples (format = 1, answer = 0):

Problem: Find the angle between two lines (in degrees).

Response (tail): Python outputs `100.30...`. $\boxed{100.30}$

Ground truth: 90°. × The model computed the obtuse angle instead of the acute angle between the lines.

Problem: Let $p(x)$ be a degree-5 polynomial with $p(n) = \frac{n}{n^2-1}$ for $n = 2, \ldots, 7$. Find $p(8)$.

Response (tail): Lagrange interpolation via SymPy gives `0.800`. $\boxed{\frac{4}{5}}$

Ground truth: $\frac{3}{56}$. × The model evaluated the constraint function $\frac{n}{n^2-1}$ directly at $n = 8$, conflating it with the interpolated polynomial p.

Problem: $\sin D = 0.7$ in a right triangle. Find DE.

Response (tail): $DE = \sin D \times DF = 0.7 \times 7 = 4.9$. $\boxed{4.9}$

Ground truth: $\sqrt{51}$. × The model multiplied by the wrong side; $\sin D$ = opposite/hypotenuse, not leg/leg.

Category 3 examples (format = 0, answer = 0):

Problem: $2 \cdot 3 \cdot 4 \cdot 5 + 1 = 121$; how many distinct values can be obtained by inserting parentheses?

Response (tail): The model generates an enormous Python list literal enumerating all parenthesization indices (`expr[i+183], expr[i+184], ...`), consuming all 2048 tokens before reaching a conclusion or producing `\boxed{}`. Parser returns `format_reward=0`.

Problem: What power of 4 equals 8? Express as a common fraction.

Response (tail): "$\ldots x = \log(8)/\log(4) \approx 1.25$, which can be expressed as the common fraction 5/4."

Ground truth: $\frac{3}{2}$. The model answers in prose without `\boxed{}`, and also computes the wrong answer ($\log_4 8 = \frac{3}{2}$, not $\frac{5}{4}$).

Problem: Seven bags of gold coins; find the minimum coins per bag so all eight bags are equal after receiving 53 extra coins.

Response (tail): "$\ldots x = 24.428\ldots$; rounding up gives 25, so the total is 203."

Ground truth: 203. The model arrives at the *correct* numerical answer but writes it in prose rather than `\boxed{203}`, so the parser returns `format_reward=0`. This is a **false negative**—the reward function penalises a correct answer solely due to a formatting omission.

Analysis of Category 3 (format = 0): The issue lies *predominantly with the base model, not the parser*. The `question_only_reward_fn` is intentionally lenient—it only requires a single `\boxed{...}` anywhere in the output. The 30 format failures fall into two patterns:

1. **Token-limit blowups** (∼20 cases): On hard combinatorics or algebra problems, the base model generates enormous intermediate code blocks that consume all 2048 tokens before producing a conclusion or `\boxed{}`. This is a failure of the base model to produce concise reasoning, not a parser limitation.
2. **Prose answers without boxing** (∼10 cases): The model occasionally delivers its answer in natural language without the required `\boxed{}` wrapper. Notably, at least one such case (gold-coins, index 50) is a **false negative**—the model's prose answer is numerically correct (203) but undetectable by the grader.

In no observed case did the parser fail to extract a `\boxed{}` that was genuinely present in the output.

Analysis of Category 2 (format = 1, answer = 0): The 166 wrong-answer cases represent genuine reasoning errors. Common patterns include:

1. **Geometric/trigonometric setup errors**: The model applies a formula to the wrong sides or angles.
2. **Buggy code trusted blindly**: Many responses use inline Python. In several cases the code contains a conceptual bug, yet the model treats the erroneous output as its final answer.
3. **Domain/constraint oversights**: The model sets up the correct algebraic framework but ignores a feasibility constraint, arriving at a numerically plausible but incorrect result.

Part (b): Zero-shot Baseline Performance

Solution

Qwen 2.5 Math 1.5B achieves **60.8% accuracy** (304/500) on the MATH test set under zero-shot prompting with temperature 0 and a maximum generation length of 2048 tokens. This is a surprisingly strong baseline for a 1.5B parameter model evaluated zero-shot, reflecting the model's extensive mathematical pre-training, though it leaves substantial room for improvement via SFT and RL fine-tuning.

Problem 2 (SFT): Supervised Fine-Tuning

(a) `eval/intellect_accuracy` (b) `eval/math_accuracy`

Figure 1: SFT validation accuracy vs. evaluation checkpoint (`eval_step`) for five dataset sizes (lr = 2e-5, effective batch 224, warmup 0.1, 35 epochs). Evaluations are placed at equally-spaced intervals through training (0 = pre-training, 1 = $\frac{1}{3}$, 2 = $\frac{2}{3}$, 3 = end), so the axis reflects the same fraction of training completed for all runs. (a) `sft-full` peaks at ≈68.5% at eval_step 1 then declines; `sft-1024` rises steadily throughout; smaller datasets fall below baseline mid-training. (b) `sft-1024` is the best and still rising at eval_step 3; `sft-full` peaks around eval_step 2 then declines.

Dataset size	Steps	Val Intellect	Val Math	Test Intellect	Test Math
128	20	0.588	0.622	0.642	0.616
256	40	0.614	0.626	0.680	0.630
512	80	0.630	0.636	0.656	0.628
1024	160	0.646	**0.660**	0.664	**0.660**
full (∼10k)	1562	0.652	0.648	**0.694**	0.648

Table 2: SFT results at end of training. Baseline (untrained) accuracy is ≈0.628 on both tasks. Best per column in bold.

Solution

All runs share identical hyperparameters (lr = 2e-5, effective batch size 224, 35 epochs, warmup 0.1), and total training steps are scaled proportionally to dataset size so that every training example is seen the same number of times across all conditions. The x-axis in Figure 1 is the evaluation checkpoint index (`eval_step`): 0 is the initial pre-training evaluation, and evaluations 1, 2, 3 are placed at equally-spaced intervals through training ($\frac{1}{3}$, $\frac{2}{3}$, and 1× total steps respectively). Because total steps scale with dataset size, the same `eval_step` value corresponds to the same fraction of training completed across all runs, enabling a direct comparison of learning dynamics.

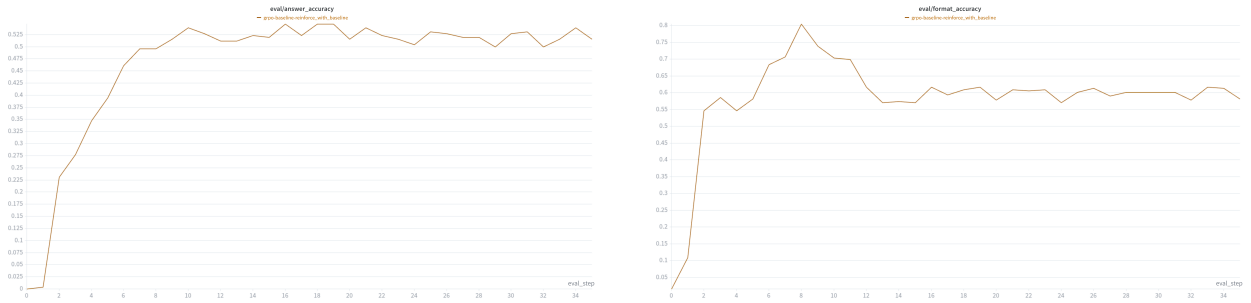
Intellect accuracy (Figure 1a) is broadly monotonic with dataset size. `sft-full` peaks at ≈68.5% at eval_step 1 ($\frac{1}{3}$ through training) but then declines, ending at 65.2% validation and **69.4% test accuracy**. `sft-1024` rises steadily throughout training, reaching 64.6% val and 66.4% test. The smaller runs (N = 128, 256) decline well below the baseline mid-training before partially recovering, consistent with overfitting to a small dataset followed by partial re-generalisation.

Math accuracy (Figure 1b) tells a more nuanced story. `sft-1024` is the best-performing run (0.660 val, **0.660 test**) and its curve is still rising at eval_step 3, suggesting it has not yet converged. `sft-full` peaks at ≈0.656 around eval_step 2 then drops to 0.648, indicating that continued training on the full dataset introduces some forgetting of mathematical reasoning. The smaller runs all dip substantially below baseline before partially recovering.

Best model: `sft-full` achieves the best test accuracy on intellect (**69.4%**), while `sft-1024` is the

strongest on math (**66.0%**). Both exceed the zero-shot baseline of ≈62.8%. We select `sft-full` as the overall best model given its higher intellect accuracy, noting that more epochs of `sft-1024` may close the gap on math.

Problem 3: GRPO Train Loop



(a) `eval/answer_accuracy` (b) `eval/format_accuracy`

Figure 2: Validation curves over 350 GRPO steps (`reinforce_with_baseline`, lr = 3e-5, rollout batch 64, group size 8). The x-axis is `eval_step`, which increments every 10 training steps (`eval_interval` = 10). (a) Answer accuracy rises rapidly from ≈0 to ≈50% within the first 6 eval steps (≈60 training steps), then plateaus around 50–53%. (b) Format accuracy peaks at ≈80% around eval_step 8 (≈80 training steps), then settles to ≈60% for the remainder of training.

Solution

Training with GRPO on the Countdown task produces rapid improvement in validation accuracy, rising from ≈0% at initialisation to ≈50% within the first 6 eval steps (≈60 training steps), and stabilising around 50–53% for the remainder of 350 steps (Figure 2a). The final test accuracy is **49.5%**. Format accuracy (b) peaks at ≈80% around eval_step 8 (≈80 training steps), then settles to ≈60%, reflecting the model trading some formatting compliance for stronger answer accuracy as training progresses.

Example rollouts over training. We evaluate the same five problems at checkpoints step_10 (early), step_90 (mid), and step_180 (late) using greedy decoding.

Early training (step 10): The model has not yet learned the `<answer>` format or the number-usage constraint. On $\{3, 7, 12\} \to 16$:

```
Step 1:  12 + 3 = 15   Step 2:  15 + 1 = 16
<answer> 12 + 3 + 1 = 16 </answer>
```

The format tag is present but the equation uses `1`, which is not in the input list. On harder problems the model loops indefinitely or fails to produce `<answer>` at all (format reward = 0). Mean answer reward at step 10: **0/5**.

Mid training (step 90): The model has learned the format and solves easy-to-medium problems. On $\{3, 7, 12\} \to 16$:

```
Step 1:  12 + 7 = 19   Step 2:  19 - 3 = 16
<answer> 12 + 7 - 3 = 16 </answer>
```

and on $\{2, 4, 6, 8\} \to 48$:

```
(8 - 4) * 6 * 2 = 48   <answer> (8 - 4) * 6 * 2 = 48 </answer>
```

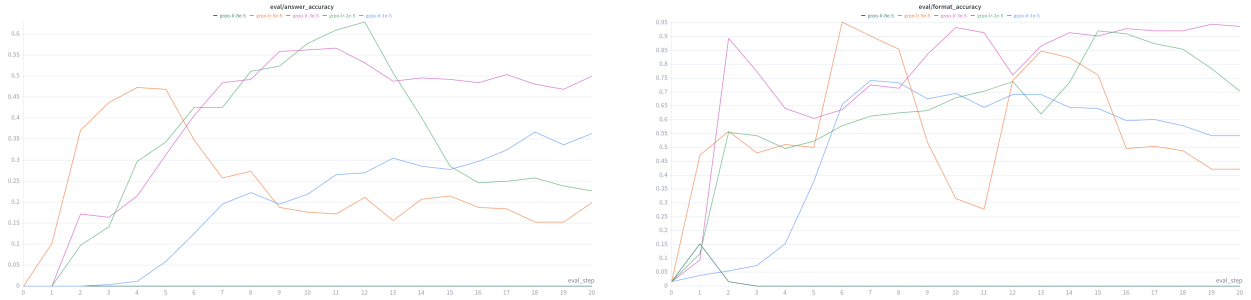
It fails $\{2, 5, 10, 25\} \to 100$, producing an expression that reuses a number (`2*(5*10-25)*2=100` uses 2 twice). Mean answer reward at step 90: **3/5**.

Late training (step 180): The model now backtracks when a chain fails. On the hardest problem, $\{11, 13, 17\} \to 7$:

```
Step 1:  13 - 11 = 2   Step 2:  2 + 17 = 19   [dead end]
Step 1:  13 - 17 = -4   Step 2:  -4 + 11 = 7
<answer> 13 - 17 + 11 = 7 </answer>
```

It still fails $\{2, 5, 10, 25\} \to 100$, which requires a two-step multiplicative structure ($10 \times 5 \times 2 = 100$). Mean answer reward at step 180: **3/5**.

Problem 4: Learning Rate Sweep



(a) `eval/answer_accuracy` (b) `eval/format_accuracy`

Figure 3: LR sweep validation curves over 200 GRPO steps (rollout batch 64, group size 8, `reinforce_with_baseline`, `masked_mean`). (a) 3e-5 rises steadily and plateaus around 47–50%; 2e-5 peaks highest (≈62%) but then crashes; 5e-5 collapses after an early spike; 1e-5 rises slowly but never diverges; 8e-5 is near zero throughout. (b) 2e-5 achieves the highest format accuracy (>90%) after its answer accuracy crashes—the collapsed policy defaults to producing formatted but incorrect outputs; 5e-5 suffers a dramatic mid-training format collapse (≈28% at eval_step 10); 3e-5 maintains stable formatting around 70–75%; 1e-5 acquires formatting slowly (≈54% at eval_step 20).

LR	Val Acc @ 200	Test Acc	Train Reward	Grad Norm
1e-5	0.363	0.375	0.428	1.02
2e-5	0.227	0.215	0.253	43.4
3e-5	**0.500**	**0.419**	**0.472**	**13.98**
5e-5	0.199	0.156	0.228	0.30
8e-5	0.000	0.000	0.003	0.00

Table 3: LR sweep results at 200 steps (rollout batch 64, group size 8, `reinforce_with_baseline`, `masked_mean`). All metrics are final values logged at step 200.

Solution

We swept five learning rates {1e-5, 2e-5, 3e-5, 5e-5, 8e-5}, all trained for 200 steps with rollout batch 64 under identical hyperparameters.

lr = 3e-5 achieves the best final performance, reaching 50.0% validation accuracy and 41.9% test accuracy (Table 3). Its validation curve rises steadily and plateaus stably around 47–50%, with a moderate gradient norm of ≈14 indicating healthy but not excessive updates.

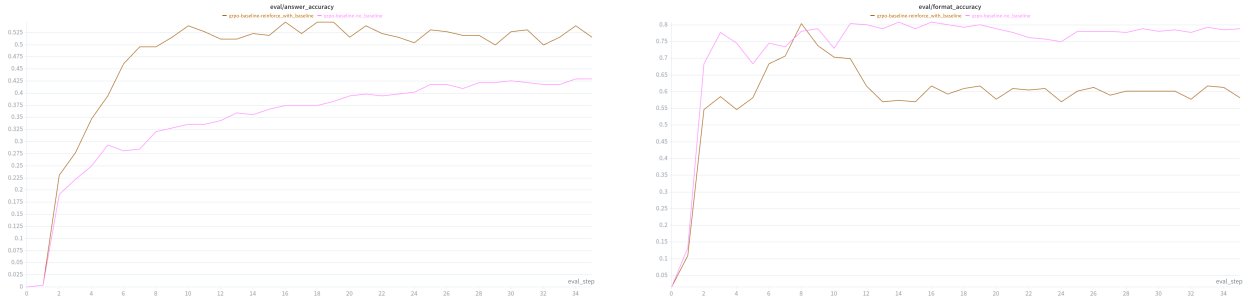
lr = 2e-5 exhibits the most striking behaviour: it actually achieves the *highest peak* validation accuracy of the sweep (≈62% at eval_step 12), but then crashes back to ≈23% by eval_step 20, yielding a final test accuracy of only 21.5%. The spike in gradient norm to 43.4 points to instability—the policy temporarily finds a good region but then overshoots and collapses. **lr = 5e-5** shows a similar pattern on a shorter timescale: a rapid rise to ≈47% by eval_step 5 followed by a crash, ending at 15.6% test accuracy. The near-zero final gradient norm (0.30) indicates the policy has become degenerate by eval_step 20.

lr = 1e-5 is the most stable run—the validation curve is monotonically increasing and never reverses—but it learns slowest, reaching only 37.5% test accuracy. **lr = 8e-5** diverges completely from eval_step 1, with near-zero accuracy and gradient norm throughout.

The pattern across learning rates reveals a clear stability–speed tradeoff: lower LRs are more stable but slower, while higher LRs learn faster initially but are prone to catastrophic divergence. lr = 3e-5 sits at the sweet spot. We use **lr = 3e-5** for all subsequent experiments.

The format accuracy curves (Figure 3b) provide additional insight into the failure modes. lr = 5e-5 suffers a dramatic mid-training format collapse (≈28% at eval_step 10) before partially recovering to ≈43%—its answer accuracy drop is driven by losing formatting ability, not mathematical reasoning. Counterintuitively, lr = 2e-5 achieves the highest format accuracy of the sweep (>90%) after its answer accuracy crashes—the degenerate policy has settled into producing well-formatted but arithmetically incorrect outputs. lr = 3e-5 maintains stable formatting around 70–75% throughout, consistent with balanced learning of both format and answer correctness. lr = 1e-5 acquires formatting slowly (≈54% at eval_step 20).

Problem 5: Effect of Baselines



(a) `eval/answer_accuracy` (b) `eval/format_accuracy`

Figure 4: Validation curves over 350 GRPO steps for `reinforce_with_baseline` vs. `no_baseline` (lr = 3e-5, rollout batch 64, group size 8, `masked_mean`, `use_std_normalization=True`). (a) `reinforce_with_baseline` rises rapidly and plateaus around 50–53%; `no_baseline` rises slowly but steadily and has not fully plateaued by step 350. (b) `no_baseline` achieves higher and more stable format accuracy (≈78%) throughout; `reinforce_with_baseline` rises early then settles lower (≈60%).

Loss type	Val Acc	Test Acc	Val Format Acc	Grad Norm
`reinforce_with_baseline`	0.516	**0.495**	0.582	59.1
`no_baseline`	0.430	0.423	0.789	0.200

Table 4: Baselines comparison at step 350 (rollout batch 64, lr = 3e-5).

Solution

`reinforce_with_baseline` outperforms `no_baseline` on answer accuracy, achieving a final test accuracy of **49.5%** vs. **42.3%** (Table 4). `reinforce_with_baseline` rises very rapidly in the first ∼6 eval steps (≈60 training steps), reaching ≈50% and plateauing there. `no_baseline` rises slowly and steadily throughout all 350 training steps, still improving at the end, suggesting it has not yet converged.

The format accuracy curves (Figure 4b) reveal an interesting tradeoff: `no_baseline` achieves *higher* format accuracy (≈78%) than `reinforce_with_baseline` (≈60%). With no baseline, the large raw reward signal primarily drives the policy toward producing well-formatted `<answer>` tags, but provides a weaker signal for distinguishing correct from incorrect arithmetic. The group-mean baseline removes this easy formatting reward from the advantage, focusing the gradient signal on answer correctness—at the cost of some format compliance.

The gradient norm confirms the difference in training dynamics: 59.1 for `reinforce_with_baseline` vs. 0.200 for `no_baseline`, reflecting the much larger and noisier gradient signal under raw rewards.

Problem 6: Length Normalization (Theory)

Solution

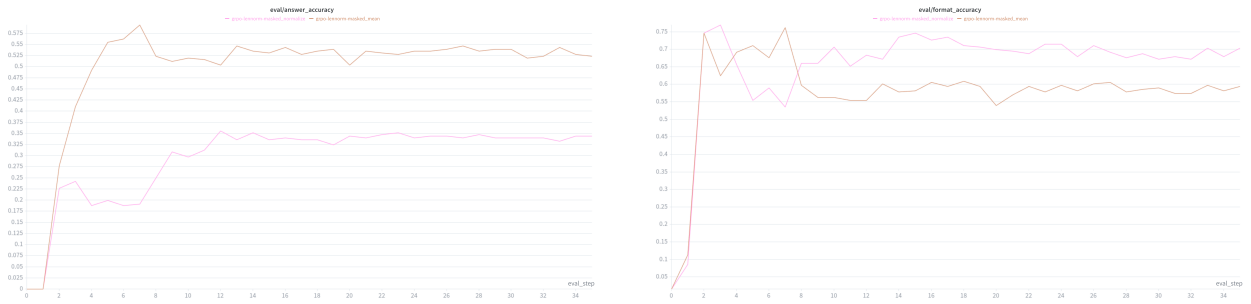
The two approaches differ in how they assign gradient credit across responses of different lengths. `masked_mean` divides the per-token loss sum by the number of response tokens in each sequence individually, so every response contributes the same effective gradient magnitude to the batch update regardless of length. The advantage is that long responses do not dominate the gradient simply due to having more tokens. The disadvantage is a *length bias in credit assignment*: a short correct response receives larger per-token gradient updates than a long correct response, implicitly incentivising the policy to produce shorter completions even when longer reasoning chains are needed.

`masked_normalize` divides by a fixed constant (typically `max_gen_len`), so each response's gradient contribution scales with its actual length. This is closer in spirit to the policy gradient derivation, which has no per-sequence length normalisation. The benefit is unbiased credit attribution with no implicit length penalty. The downside is that long responses dominate the gradient, which can cause instability when generation lengths vary widely within a batch.

Concrete example from the assignment: With a batch of two responses (4 tokens and 7 tokens, both with advantage $A = 2$ and probability ratios of 1), `masked_mean` gives each response a mean loss of 2.0—equal total weight per response. Under `masked_normalize` (constant = 7), the shorter response contributes $4/7 \approx 1.14$ and the longer contributes $7/7 = 2.0$ to the batch mean loss—the longer response has $1.75\times$ the gradient weight of the shorter one.

When each approach is preferable: `masked_mean` is the safer default when response lengths vary greatly and training stability is a concern. `masked_normalize` is preferable when response lengths are relatively uniform and unbiased credit assignment matters more than stability. For example, in a task like Countdown where successful solutions tend to have a natural, bounded length range, the length variance is smaller and `masked_normalize` is more likely to be beneficial.

Problem 7: Length Normalization (Empirical)



(a) `eval/answer_accuracy` (b) `eval/format_accuracy`

Figure 5: Length normalization validation curves over 350 GRPO steps (lr = 3e-5, rollout batch 64, group size 8, `reinforce_with_baseline`, `use_std_normalization=True`). (a) `masked_mean` rises quickly and plateaus stably around 50–55%; `masked_normalize` oscillates early then plateaus at a lower ≈33–35%. (b) `masked_normalize` maintains higher format accuracy (≈68–75%) than `masked_mean` (≈58–60%) throughout.

Length norm	Val Acc	Test Acc	Val Format Acc	Grad Norm
`masked_mean`	0.523	**0.442**	0.594	20.3
`masked_normalize`	0.344	0.258	0.703	23.1

Table 5: Length normalization comparison at step 350 (rollout batch 64, lr = 3e-5).

Solution

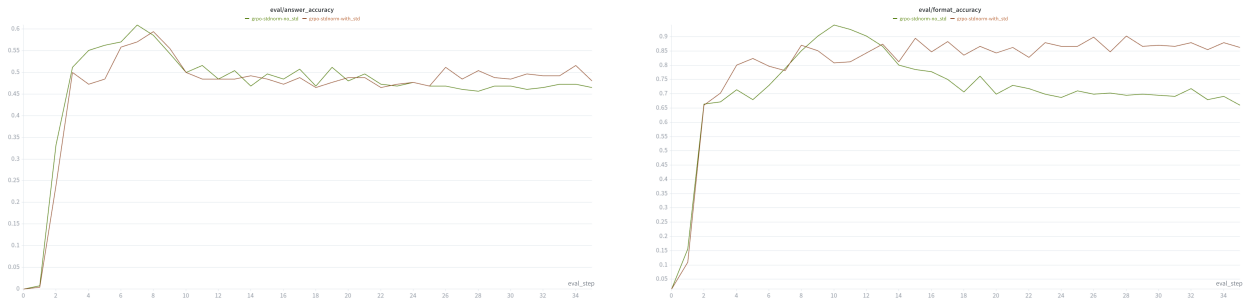
`masked_mean` substantially outperforms `masked_normalize`, achieving a final test accuracy of **44.2%** vs. **25.8%** (Table 5). `masked_mean` rises rapidly in the first ∼7 eval steps (≈70 training steps) and plateaus stably around 50–55%, while `masked_normalize` oscillates erratically in early training before settling into a lower plateau of ≈33–35% that it never escapes.

The format accuracy plot (Figure 5b) shows that `masked_normalize` actually achieves *higher* format accuracy (≈70%) than `masked_mean` (≈60%). This mirrors the pattern seen in the baselines experiment: when the gradient signal is less focused on answer correctness, the policy defaults to optimising the easier formatting objective. Under `masked_normalize`, longer responses receive proportionally larger gradient updates, which increases gradient variance across the batch and makes it harder to learn the arithmetic reasoning needed to solve Countdown—the policy learns to produce well-formatted but incorrect answers.

Notably, the gradient norms are similar for both runs (20.3 vs. 23.1), suggesting the instability of `masked_normalize` is not simply a magnitude issue but rather one of gradient direction: the length-weighted updates are less consistently oriented toward improving answer accuracy.

We fix to `masked_mean` for subsequent experiments.

Problem 8: Group Standard Deviation Normalization



(a) `eval/answer_accuracy` (b) `eval/format_accuracy`

Figure 6: Std normalization validation curves over 350 GRPO steps (lr = 3e-5, rollout batch 64, group size 8, `reinforce_with_baseline`, `masked_mean`). (a) Both runs follow nearly identical answer accuracy trajectories, peaking around 0.60 at eval_step 8 (≈80 training steps) before settling around 0.47–0.48. (b) `with_std` maintains substantially higher format accuracy (≈85%) than `no_std` (≈68%).

Std norm	Val Acc	Test Acc	Val Format Acc	Grad Norm
`with_std`	0.480	0.395	0.863	30.5
`no_std`	0.465	0.400	0.660	10.6

Table 6: Std normalization comparison at step 350 (rollout batch 64, lr = 3e-5).

Solution

Unlike the previous ablations, the two std normalization variants perform comparably on answer accuracy: both peak around 60% at eval_step 8 (≈80 training steps) before settling into a plateau of ≈47–48%, and their final test accuracies are nearly identical (39.5% vs. 40.0%, Table 6). Notably, `no_std` has a marginally higher test accuracy (40.0% vs. 39.5%), though the difference is within noise. The clearest difference lies in *format accuracy* (Figure 6b): `with_std` maintains a stable ≈85% format accuracy throughout, while `no_std` peaks around 92% early before declining to ≈68%. This suggests that dividing by the group standard deviation provides a more consistent training signal that preserves formatting behaviour, whereas without the normalisation the policy gradually deprioritises formatting as training progresses.

The gradient norm is also notably higher under `with_std` (30.5 vs. 10.6), consistent with the theoretical argument that std normalisation amplifies the gradient signal for groups with low reward variance. Despite this, the added noise does not appear to hurt answer accuracy.

Given the near-identical answer accuracy and higher format compliance, we fix to `with_std` as the default for subsequent experiments.